Assignment 1: Island Logistics

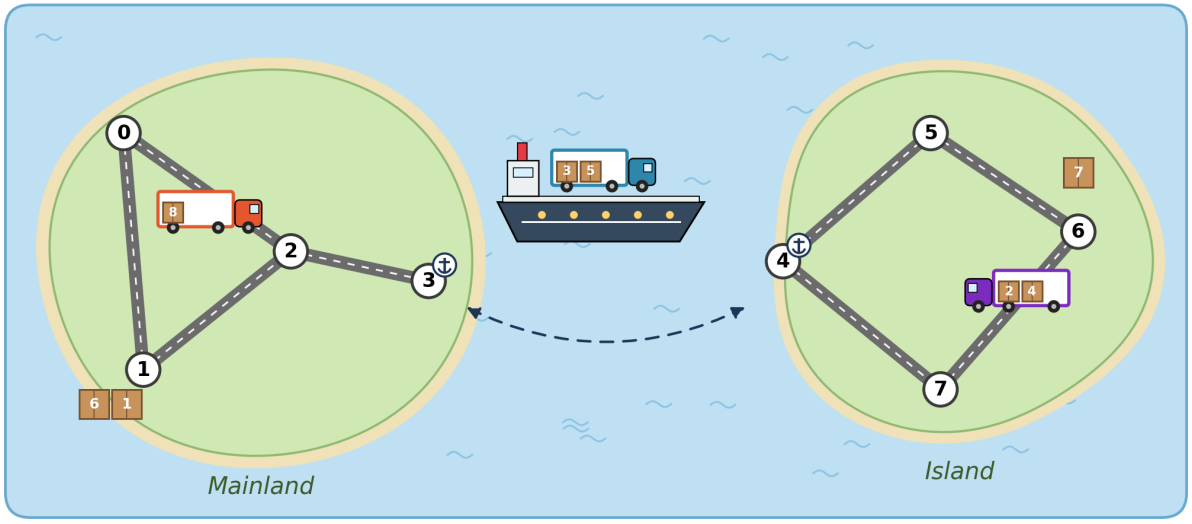
Trucks, Weighted Packages and Ferries

Bar-Ilan CS 89-570
Introduction to Artificial Intelligence

November 4, 2026

1 Introduction

In this exercise you will *experiment with solving a planning/path-finding problem* using deterministic search algorithms we studied (A*).



Summary of the Domain A logistics company delivers packages across a group of islands. On every island there are locations connected by roads, and the company's trucks drive along these roads. Each truck can carry packages up to a maximum total weight. Roads do not cross the sea: to get from one island to another, a truck must drive onto a *ferry*, sail with it, and drive off at another port. Each ferry can carry only a limited number of trucks at once. Every package has a starting location, a weight, and a destination. The task is to plan a shortest valid sequence of actions that delivers all packages to their destinations.

2 Problem Description

The map contains locations numbered $0, 1, \ldots, n-1$. Some pairs of locations are connected by *roads*. Roads are two-way: if there is a road between locations a and b, a truck may drive from a to b and from b to a. Some locations are *ports*, where ferries can dock. A problem instance contains a set of trucks, a set of ferries and a set of packages.

Trucks: You control a set of trucks. Each truck has a unique ID number and two properties:

- its current location,
- its maximum weight capacity (the total weight of the packages inside it may never exceed this number).

At any moment, a truck is either standing at a location or aboard exactly one ferry.

Ferries: You also control a set of ferries. Each ferry has a unique ID number and three properties:

- the port where it is currently docked,
- the set of ports it serves (it may sail only between these ports),
- the maximum number of trucks it can carry at the same time.

Packages: Each package has a unique ID number and three properties:

- a starting location,
- a weight,
- a destination location.

A package is either lying at some location or inside exactly one truck. Packages never board a ferry by themselves – they cross the sea only inside a truck.

Actions: Each step applies exactly one action. There are six types of actions:

- DRIVE(TRUCK_ID, LOCATION1, LOCATION2) – move a truck along a road from its current location to an adjacent location. Legal only if the truck is standing at a location (not aboard a ferry) and there is a road between its current location and the target location. All packages inside the truck move with it.
- LOAD(PACKAGE_ID, TRUCK_ID) – put a package into a truck. Legal only if the package is lying at the location where the truck is standing (the truck is not aboard a ferry), and after loading, the total weight of the packages inside the truck does not exceed the truck's maximum weight capacity.
- UNLOAD(PACKAGE_ID, TRUCK_ID) – take a package out of a truck and place it at the truck's current location. Legal only if the package is inside that truck and the truck is standing at a location (not aboard a ferry).
- BOARD(TRUCK_ID, FERRY_ID) – drive a truck onto a ferry. Legal only if the truck is standing at the port where the ferry is currently docked, and the number of trucks aboard the ferry is smaller than its maximum. The truck keeps all its packages.

- SAIL(FERRY_ID, PORT1, PORT2) – move a ferry from its current port to another port in the set of ports it serves. All trucks aboard (and the packages inside them) move with the ferry. A ferry may sail empty. Each ferry has a set of avalibel ports it can sail to.
- DISEMBARK(TRUCK_ID, FERRY_ID) – drive a truck off a ferry. Legal only if the truck is aboard that ferry. The truck is now standing at the port where the ferry is docked.

At each time step, exactly one action is executed. Every SAIL *action costs* 3*, and every other action costs* 1*.*

Action representation (string format). Let t denote a **truck ID**, f a **ferry ID**, p a **package ID**, and l a **location number**. Each action must be encoded as one of the following strings:

Action	String
DRIVE(T, L1, L2)	`DRIVE{t,l1,l2}`
LOAD(P, T)	`LOAD{p,t}`
UNLOAD(P, T)	`UNLOAD{p,t}`
BOARD(T, F)	`BOARD{t,f}`
SAIL(F, L1, L2)	`SAIL{f,l1,l2}`
DISEMBARK(T, F)	`DISEMBARK{t,f}`

For example, if truck 10, standing at location 1, loads package 30, drives to location 2, boards ferry 20 docked there, sails with it to port 3 and drives off, the output should be: [`LOAD{30,10}`, `DRIVE{10,1,2}`, `BOARD{10,20}`, `SAIL{20,2,3}`, `DISEMBARK{10,20}`].

In the successor function, each child is represented as `(action, next_state)`, and you should store this string in the `action` field. There are no spaces inside the strings.

Additional domain rules

- A truck may drive only along existing roads, one road per action. There are no roads across the sea.
- A ferry may sail only to ports in its own set of served ports. Sailing between any two of its ports takes a single action.
- Packages can be loaded and unloaded only while the truck is standing at a location – never while it is aboard a ferry.
- Packages do not change trucks automatically. To move a package from one truck to another, it must be unloaded at some location and then loaded into the other truck.
- The truck weight capacity and the ferry truck capacity must always be respected.
- A package cannot be inside two trucks, and a truck cannot be aboard two ferries, at the same time.

3 Initial State and the Goal

Initial state representation The initial state of the problem is represented as a Python dictionary with the following entries:

1. **Locations** – an integer n. The locations are $\{0, 1, \ldots, n-1\}$.
2. **Roads** – a set of pairs (a, b); each pair is a two-way road between locations a and b. Each road appears once, in one of its two directions.
3. **Ports** – the set of locations that are ports.
4. **Trucks** – a dictionary whose keys are truck IDs. The value of truck ID t is a tuple

   $$(l, w_{\max}),$$

   where l is the truck's current location and $w_{\max}$ is its maximum weight capacity. Initially, all trucks are standing at locations and are empty.
5. **Ferries** – a dictionary whose keys are ferry IDs. The value of ferry ID f is a tuple

   $$(l, L, k_{\max}),$$

   where l is the port where the ferry is docked, L is the set of ports it serves, and $k_{\max}$ is the maximum number of trucks it can carry. Initially, all ferries are empty.
6. **Packages** – a dictionary whose keys are package IDs. The value of package ID p is a tuple

   $$(l_{start}, w, l_{goal}),$$

   where l_{start} is the package's starting location, w is its weight, and l_{goal} is its destination.

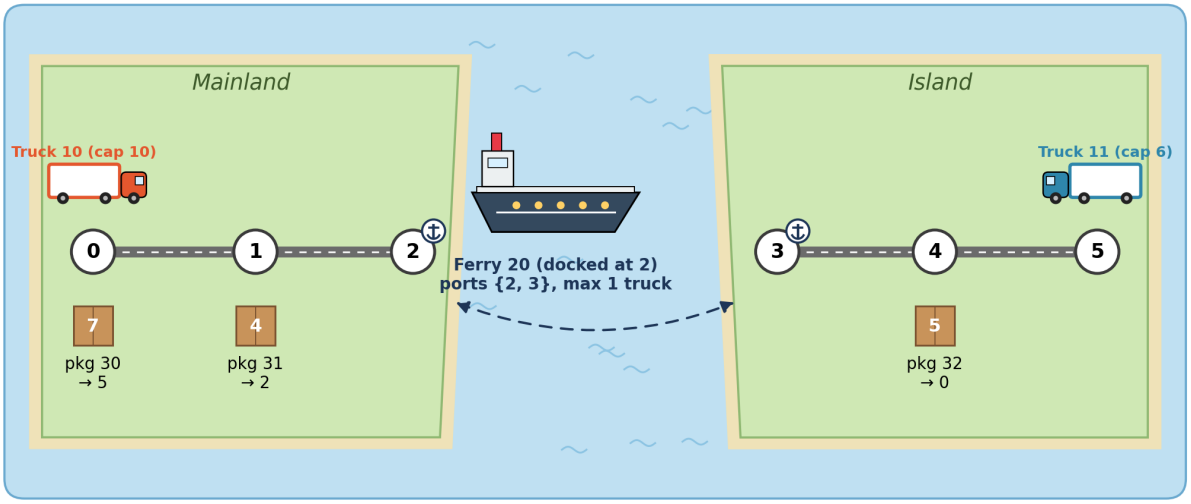
All IDs (of trucks, ferries and packages) are distinct integers.

A state must additionally encode, for each package, whether it is lying at a location or inside a truck, and for each truck, whether it is standing at a location or aboard a ferry. You are free to choose the exact internal representation, as long as it is consistent and hashable.

Goal description The problem is **SOLVED** if every package is lying at its destination location (and is not inside a truck). The final positions of the trucks and ferries do not matter.

4 Example

```
init_state_p1 = {
    "Locations": 6,
    "Roads": {(0, 1), (1, 2), (3, 4), (4, 5)},
    "Ports": {2, 3},
    "Trucks": {
        10: (0, 10),
        11: (5, 6)
    },
    "Ferries": {
        20: (2, (2, 3), 1)
    },
    "Packages": {
        30: (0, 7, 5),
        31: (1, 4, 2),
        32: (4, 5, 0)
    }
}
```



5 Tips

We provide a starting state `init_state`. Each student may represent the state however they like, but once you define the state representation in the Python initialization function, you must keep using the same representation for every state you create in the successor function.

Note that each state must be *hashable*, i.e., all its components must be immutable. Examples of immutable structures in Python include: tuple, string, and frozenset. You can also make hashable classes, but then you need to implement `__eq__` and `__hash__`.

- Keep the **A* heuristic admissible**. For example, the blind heuristic is admissible but weak. The real challenge is to make it informative and still admissible.
- Things that never change (the roads, the ports, capacities, weights and destinations) do not need to be part of the state. You may precompute on the map or on a simplified version of the problem in the constructor – for example, distances on the road network – but you may not search the real state space outside the provided search.
- Handle **truck weight capacity** and **ferry truck capacity** correctly.
- Remember that a truck aboard a ferry cannot drive, load or unload, and a ferry can only sail between the ports it serves.
- Generate only **legal actions**. This will be checked.
- Keep the **successor function** as small and efficient as possible, to avoid redundant successors. You are allowed to create only some of the successor, under the condition that your code still provides an optimal solution to the problem.
- Note that while LLMs can help you brainstorm heuristics, they can lie that heuristic is admissible.

6 Important

Since the problem can be represented in many different ways (again – you are free to choose how to represent the state and the problem), we need the action names and the IDs of the trucks/ferries/packages to be consistent among all submissions. Do not change the IDs you got as input. Use the provided action names and format. For example, the action that drives truck 10 from location 1 to location 4 is written as `DRIVE{10,1,4}`.

7 What You Must Implement

We provide a starter codebase. Most infrastructure is already implemented. Your job is to complete the methods so that the checker can solve instances.

Context. The state is your encoding, that you choose to represent the problem.

Files and required functions (in `ex1.py`)

You *must* implement the following (you may add helpers as you see fit):

1. `successor(state) -> List[(action, next_state)]`
   - **Input:** a `state` object (hashable) representing the current problem configuration.
   - **Output:** a *list* of tuples (`action`, `next_state`).
   - **Try to keep this list as small as possible by using pruning techniques, but do not prune any action that may be part of an optimal solution.**
   - **Actions to consider (6 types):** DRIVE, LOAD, UNLOAD, BOARD, SAIL, DISEMBARK.
   - **Side effects (purely functional):** `next_state` must be a *new* object with updated positions/status; do not mutate `state` in place.
2. `goal_test(state) -> bool`
   - **Input:** a `state`.
   - **Return:** `True` iff every package is lying at its destination location and is not inside any truck.
3. `h_astar(node) -> int` (*A* heuristic; must be admissible*)
   - **Input:** a `node` whose property `node.state` holds the current state.
   - **Output:** a nonnegative integer that *lower-bounds* the remaining cost to any goal.
   - **Meaning:** this is the heuristic A* uses; it must never overestimate the optimal remaining cost. (Consistency is preferred but not required.)

8 Scoring & Evaluation (How the checker works)

- Each instance is tested once with **A***, using your implementations of `successor`, `goal_test`, and the corresponding heuristic. The running time of each instance is measured, and the time limit includes the constructor of your problem class.
- **Easy set (50 points):** 10 instances, given to you in advance. Each instance has a 30-second time limit, and solving it (i.e., outputting an optimal plan) within the limit earns you 5 points.
- **Hard set (35 points):** 7 instances, given to you in advance. The same rules apply, and solving an instance earns you 5 points.
- **Hidden set (15 points):** C instances that are *not* given to you, so that solutions do not overfit to the easy and hard sets. Each instance is worth up to $15/C$ points and has a 45-second time limit. Your score on an instance depends on your running time, by groups (for example, under 5 seconds, under 15 seconds, under 30 seconds, and so on): a faster group earns more points, and the fastest group earns the full $15/C$. This is not a competition: you are not ranked against other students, and everyone who solves all the instances in the fastest group gets the full 15 points, so every student can reach 100. We will decide the exact groups after we see the results.

What counts as "solved"? An instance is considered solved if your algorithm returns a *valid* action sequence that reaches a goal state and passes the automatic simulator checks (preconditions/effects and goal).

- **Additional requirement for A*:** the returned plan *must be optimal*, i.e., have the minimum possible total cost. The checker verifies optimality.

Code checks.

- Your code will be checked by an AI. You may not change `path_cost` or the search algorithm, and you may not write your own priority queue or search.
- The tester changes the names of the problems and hashes them before checking them. Do not try to detect which problem is running and return a stored answer.
- We will check your heuristic on other problems. A non-admissible heuristic will cost you 30 points from your total score.

9 Attached files

The starter code includes the following files:

1. **`ex1.py`** — *the only file you submit and the only one graded.* Implement `successor`, `goal_test`, `h_astar`. The method `path_cost` is already implemented; do not change it.
2. **`ex1_check.py`** — local *self-checker*. Use it to run your code on the provided instances. You may add *more* problems here for your own testing, but the grading system *will not* use your added problems. Changes to this file do not affect grading.
3. **`ex1_maker.py`**: optional tool that draws a problem so you can see it. You do not submit this file.
4. **`ex1_gui.py`**: optional tool that runs your `ex1.py` on a problem and plays the plan it returns. You do not submit this file.
5. **`utils.py`** and **`search.py`** — auxiliary code (data structures and search drivers used by the assignment). You do not submit these files; do not rely on modifying them for your solution. You are not allowed to change the search algorithm or the priority queue, or to replace them with your own, in any way.

How to self-check (quick start). Run the checker from the assignment folder:

```
python ex1_check.py                 # all the problems
python ex1_check.py X               # one set: X, Y or examples
python ex1_check.py x3              # one problem
python ex1_check.py p1 x3 y2        # problems of your choice
python ex1_check.py --list          # the names of all the problems
python ex1_check.py x3 --plan       # also print the plan
python ex1_check.py X --limit 60    # another time limit, in seconds
```

You can run it with python3 if you want to.

Seeing the problem and the plan. Two optional tools let you see what is happening. Nothing about them is graded.

```
python ex1_maker.py                # starts with p1
python ex1_gui.py                  # starts with p1
python ex1_maker.py x3             # starts with x3
python ex1_gui.py x3               # starts with x3
```

The two tools take exactly the same values: nothing, or the name of one problem from `python ex1_check.py --list`. Unlike the checker, they do not take a set or several problems. You can also choose a problem from the list inside the window. Just run `ex1_maker.py` and you can see all the problems: choose one from the list and it is drawn at once. You can also draw new problems: type or paste your own, or change the text of a problem, then press Draw. `ex1_gui.py` runs your `ex1.py` on a problem you choose, then plays the returned plan: press Solve, then Play. You can play it, step through it, or click any action in the list. Both tools need only Python's `tkinter`. On some Linux installs `tkinter` must be installed separately (for example `sudo apt install python3-tk`).

10 Rules & Submission

- **Edit only `ex1.py`.** Do not modify or submit any other file.
- **Libraries.** You may use `numpy` in `ex1.py`.
- **Submission.**
  1. Rename `ex1.py` to `ex1_id.py`, where `id` is your ID number. For example: `ex1_000000000.py`.
  2. Submit the renamed file using the Bar-Ilan `submit` system: `https://submit.cs.biu.ac.il/cgi-bin/welcome.cgi`
- **Individual work.** Discussing ideas is allowed, but *sharing or copying code is prohibited.*
- **Use of AI / online sources.** You may use them, but you *must* clearly state *at the top of the document* which LLMs or online sources you used and how you used them (for example: writing parts of the code, suggesting specific actions, providing ideas, or anything else). Failure to disclose this information will result in a grade reduction.
- **Academic integrity.** We take plagiarism seriously.
- **Questions.** Ask during office hours or in the assignment forum only.
- **Extensions.** No extensions will be granted except for documented exceptional circumstances via email.
- **Deadline:** 26/11/2026.